



Comparison of Logical Execution Time Programming Languages

Sid Ahmed Issolah, Marc Boyer, Damien Chabrol, Guillaume Phavorin

► To cite this version:

Sid Ahmed Issolah, Marc Boyer, Damien Chabrol, Guillaume Phavorin. Comparison of Logical Execution Time Programming Languages. 13th European Congress of Embedded Real Time Systems (ERTS), Feb 2026, Toulouse, France. <10.82331/ERTS.2026.16>. <hal-05513534>

HAL Id: hal-05513534

<https://hal.science/hal-05513534v1>

Submitted on 19 Feb 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

Comparison of Logical Execution Time Programming Languages

Sid Ahmed Issolah^{1,2}, Marc Boyer², Damien Chabrol¹, and Guillaume Phavorin¹

IAsterios Technologies, Massy, France

[sid-ahmed.issolah](mailto:sid-ahmed.issolah@asterios-technologies.com), [damien.chabrol](mailto:damien.chabrol@asterios-technologies.com), [guillaume.phavorin](mailto:guillaume.phavorin@asterios-technologies.com) @asterios-technologies.com

²ONERA/DTIS, Université de Toulouse, Toulouse, France

[sid-ahmed.issolah](mailto:sid-ahmed.issolah@onera.fr), [marc.boyer](mailto:marc.boyer@onera.fr) @onera.fr

ABSTRACT

Logical execution time (LET) paradigm received substantial emphasis from the real-time community due to its many benefits, specifically its time determinism guarantees. Therefore, many programming languages have made the choice to implement this paradigm. This raises the question of which language needs to be considered. Our paper tackles this question by exploring and comparing LET-based programming languages through a dual-lens approach: (1) global framing; which consists of chronological and contextual comparisons; and (2) the extent of pattern support capabilities. Our aim is to offer some guidance to developers in selecting the most suitable programming language based on system constraints.

Index Terms—Real-time systems, programming languages, logical execution time.

Category—Regular paper

1. INTRODUCTION

Real-time systems are those in which the correctness of system behavior depends not only on the logical results of computations but also on the physical time at which these results are produced [1]. This definition establishes an indissociable link between time determinism and functional determinism. As a result, the development of programming models that provide a clear specification of temporal behavior, in line with a correctness-by-construction approach, has become essential. The former adopted solutions have been synchronous programming languages such as Lustre [2], Esterel, and Signal [3]. However, they may contain cyclic dependencies, which require fixed-point analysis to verify their correctness [4], [5]. To avoid this, the OASIS project [6] discarded synchronous communications and refined the abstraction proposed by synchronous languages. Rather than instantaneous logical execution, it abstracted the physical execution into a logical window—the same approach proposed by Giotto [7] and later formalized as the Logical Execution Time paradigm [5].

The Logical Execution Time (LET) paradigm receives significant attention due to its salient benefits, among them:

- Decoupling application design from execution architecture. This allows software engineers to focus on system logic without being concerned about platform-specific constraints, which has become a must because of the rapid increase in software complexity—driven by the ongoing shift from hardware to software, the new trend in the automotive industry [8].
- Time determinism, here, refers to the fact that communication happens only at predefined time instants, and dataflow determinism, means that a consumer job always reads a value from the same producer job [9]. It has been revealed to be an appealing solution of communication mechanism in distributed and multi-core platform [10] for safety-critical real-time systems.

Contribution. We examine multiple LET-based languages through a structured comparison framework. Two key aspects will be discussed: (1) **expressiveness**, the ability to support some common behavior patterns (modes, periodicity, sporadicity, etc.). (2) **framework**, assessing tool-chain capabilities such as debugging, scheduling, visualization, tracing, and verification; and (3) **industrial use cases**, examining how these languages are adopted in practice and the types of systems they target.

The objective of this paper is to design a cartography to guide developers in selecting suitable LET implementations for real-time systems.

Outline. Section 2 introduces various LET-based programming languages and explains the LET model. Section 3 discusses the methods and procedures for comparing these languages, and we draw our conclusions in Section 4.

2. LOGICAL EXECUTION TIME PARADIGM

Before presenting programming languages based on LET model, we briefly introduce this paradigm.

2.1. LET Overview

Giotto was the precursor of the Logical Execution Time (LET) paradigm. It was formalized as LET at a later stage. LET is a real-time abstraction model developed for real-time control applications [7]. It defines a LET window, a time interval, for each task, such that the task response appears *as if* it takes exactly the same amount of time as the LET window, regardless of the physical execution. LET communication behavior is

Sid Ahmed Issolah. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0, which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

governed by the *Visibility Principal*. Data produced by a producer job becomes visible only at the end of its LET interval. A consumer job may read data only if it is visible before or at the start of its LET interval. That is, the task fetches (reads) the inputs at the release date (the beginning of the window). After finishing its real execution, the data is committed (but this update is hidden from other tasks) and will be pushed (published) at the termination date (the end of the window). Figure 1 illustrates the LET paradigm.

LET abstraction ensures temporal determinism, dataflow determinism, and platform independence. This means that the same LET design behaves identically on different platforms, which makes the paradigm valuable for real-time programming languages.

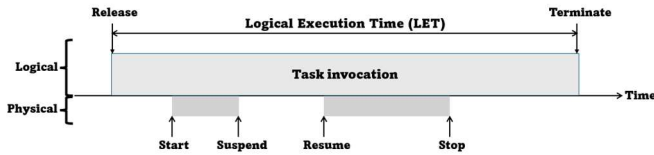


Figure 1. Logical Execution Time

To show the data flow determinism, Figure 2 illustrates two communications scenarios between a producer and a consumer task using either LET or Bounded Execution Model (BET) [4]. In BET model, data produced by a job is immediately visible at the end of its execution. Thus, communication using the BET model depends on the low-level scheduling and we see that in Figure 2, where the second instance of the consumer reads different instances of the producer depending on the start of its execution. Contrary to the BET mode, LET preserves the same behavior regardless of its physical execution, enhancing determinism.

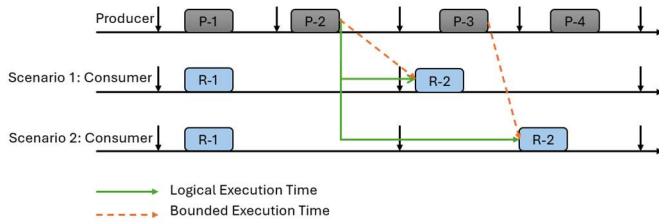


Figure 2. LET vs BET model.

2.2. LET-based programming languages

2.2.1. Giotto

Giotto, introduced in 2001, is a time-triggered, platform-independent programming language designed for hard real-time control applications on distributed platforms [11], particularly in safety-critical contexts, the first one to implement the LET paradigm. It defines software architecture that specifies both timing and functionality of the system which abstracts hardware-specific details like performance and scheduling [7].

Giotto was inspired by the Timed Triggered Architecture (TTA) [4], which realizes a time-triggered paradigm for safety-critical distributed system [7]. However, TTA is based on WCET task, which may vary depending on the hardware architecture and communication protocols, whereas Giotto claims to provide a separation of logical correctness concerns (functionality and timing) from physical realization concerns (mapping and scheduling). Thus, making a Giotto program entirely platform independent [11].

A Giotto program consists of modes, with one single mode active at a time. Each mode has a specific period and contains: (1) **mode ports**: ports are memory slots used for communication between tasks, sensors and actuators and mode switching logic using *drivers*. Inspired by physical drivers (computer drivers), a Giotto driver represents the interface between the logical program and the physical environment. It performs the actual input/output operations according to the time-triggered schedule of the mode. A driver is a function that transfers values from a source port to a destination port and may be guarded [11]. A *guard* is a Boolean expression evaluated when the scheduler considers executing the driver. A guarded driver executes only if its guard evaluates to true. (2) **Actuator updates** and **mode switch**: Actuator drivers update physical actuators [7], mode switch transitions the program from one mode to another. It has a frequency and a mode driver [11]. (3) **Tasks invocation**: task is the basic unit of Giotto which includes a set of input, output and private ports, period and a function implemented in any programming language. The task only must be started and be finished some time during its period and is updated logically at the beginning and the end of the task's period, respectively.

Giotto programs are fully time-triggered LET programs. xGiotto [12] extends Giotto with event-triggered features, but is not covered here.

2.2.2. Hierarchical Timing Language (HTL)

HTL, introduced in 2002 [13], is a real-time coordination language for hard real-time distributed control systems. As a successor of Giotto, HTL relies on the LET paradigm. It uses a tree-like structure composed of four elements [14]: (1) **tasks**: LET tasks implemented in any programming language, with the same features as in Giotto. Unlike Giotto, HTL introduces communicators as an additional communication mechanism.

Communicators are typed variables accessible only at specific periodic time instances. HTL's communication model is centered around communicators: tasks read from communicators, computers, and update communicators. The read and write instances specify the LET for the tasks: the latest read instance determines the release time, and the earliest written instance determines the termination time, as depicted in Figure 3. (2) **Modes**: in HTL, a network of concurrent tasks that shares the same periods form a mode with the same period as the tasks. Tasks with different periods, from different modes, must only communicate through communicators; tasks within the same mode can either communicate through communicators or ports. (3) **Modules**: a set of sequential modes forms a

Module. In other words, inside a module, only a single mode is active at a time. However, modules are executed in parallel (i.e., modes from different modules are concurrent). Modules dispose of communicators, and mode-switching logic. (4) The **program** block is the root node containing the set of modules executed in parallel.

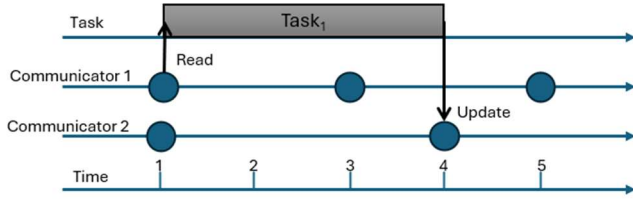


Figure 3. HTL task and communicators inspired by [14]

HTL aims to improve succinctness while keeping time-safety checking easy [5]. To avoid specifying all behavior through mode switching, HTL introduces mode refinement, allowing a mode to refine (specialize) a program block into concrete modes [15]. This enables hierarchical configuration.

HTL refinement supports modular programming: if parts of a refinement change, only the abstract program of the modified refinement must be rechecked, not the entire program [5], [14].

2.2.3. Timing Definition Language (TDL)

TDL was developed as part of project MoDECS at the Paris Lodron University of Salzburg [16]. Introduced in 2004, it is another successor of Giotto and employs LET abstraction to specify platform-independent execution times for periodic, time-triggered tasks [17]. As in Giotto, a TDL application consists of sensors, actuators, and modes, each containing periodic task invocations and other periodic activities (actuator updates, mode-switch checks). Like HTL, TDL groups them into modules, but communication differs: TDL does not use communicators, and unlike Giotto, mode switch is not based on drivers [8]. A TDL program consists of parallel modules, each with a required start mode—a concept absent in HTL and Giotto. TDL also prohibits a mode switch unless all tasks in the mode have completed their executions [18]. Overall, TDL and Giotto are conceptually similar. The main distinction is that TDL provides a tool chain that automates aspects that were manual in Giotto, especially communication, which explains some of TDL’s added restrictions.

Extended TDL (E-TDL) [19], [20] introduces offsets and LET windows shorter than task periods, unlike Giotto and TDL, where LET equals the period. E-TDL is not covered here.

2.2.4. Lingua Franca

Lingua Franca (LF) is a reactor-oriented coordination language introduced in 2020 through a collaboration between UC Berkeley, TU Dresden, UT Dallas, and Kiel University [21]. LF is a polyglot coordination language intended as “a bridge language” [22]. Its design principles are lean coordination,

polyglot (a flexible multi-language wrapper that focuses on the user’s language and requires only minor adaptation effort [23]), concurrency, explicit time, and determinism.

Computations in LF are encapsulated in *reactors* [17], which communicate through typed *ports* and contain *reactions*—target-language code blocks (C, C++, Python, Java, or Rust) that execute only when triggered [21]. *Reactions* respond to events produced by *timers* (periodic events) or *actions* (irregular, program-scheduled events), and each event is assigned a *tag*, a logical timestamp (time, microstep) that orders events deterministically using superdense time [24]. This model ensures that reactions run logically instantaneously at their tag, enabling parallel execution without races or deadlocks.

Like HTL, LF supports hierarchical design, but through composite reactors (reactors containing other reactors). The main reactor (program entry point) cannot have input/output ports [21].

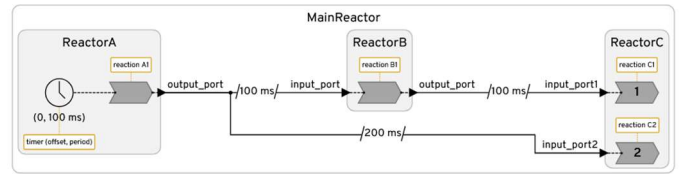


Figure 4. An example of LF program [25]

Figure 4 illustrates a simple LF program composed of three reactors arranged in a pipeline. ReactorA generates periodic events via a timer and forwards them to ReactorB after a declared logical delay. ReactorB processes each event in its reaction and sends the result to ReactorC, again with a specified delay. ReactorC contains two reactions, C1 and C2, which process the incoming data sequentially because they may share state.

Although LF is not built on LET but on the synchronous-reactive (SR) approach [26], it also implements LET behavior using tags: tasks execute logically instantaneously, but outputs are delayed [24].

LF also implements modes to support state-machine behavior. Modes can declare local contents (state variables, reactions, timers), except ports, which belong to reactors. As in TDL, an LF program requires a start mode. Unlike HTL, LF modes cannot be nested; hierarchy is achieved through reactor composition.

2.2.5. Parallel Synchronous C (PsyC)

PsyC is an industrial programming language dedicated to real-time safe-critical real-time systems, currently developed by Asterios Technologies. Unlike the previously described programming languages, PsyC introduces what we call the Synchronous Logical Execution Time (sLET) paradigm.

2.2.5.1 sLET paradigm

sLET inherits from both the LET and SR paradigm. It retains both their benefits such as functional and temporal determinism, platform independence [27].

As in LET and SR, logical and physical times serve different purposes: logical time expresses high-level temporal requirements, while physical time reflects execution on hardware [28]. sLET is based on logical time. Each logical clock has its own period, and the programming language references directly to those logical clocks.

The formal foundation of the sLET paradigm was defined in [27], providing a semantic framework linking LET with synchronous clocks. LET window boundaries—earliest start and latest finish—are defined by Time Synchronization Points (TSPs) [29], which structure the execution of all tasks in a globally synchronized timeline, see Figure 5.

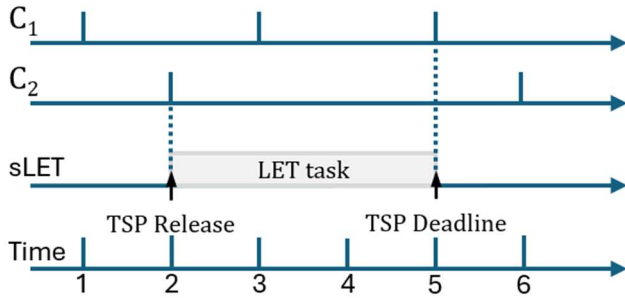


Figure 5. sLET interval (the activation date, respectively termination date, is defined on Clock C_2 , resp. C_1) [28].

2.2.5.2 PsyC language description

A program in PsyC consists of Agents which are an infinite sequence of Elementary Actions (EA) where an EA is a computation that executes in a LET window [30]. The termination date of an sLET interval corresponds to the release date of the next interval for that task. Unlike the LET model, sLET does not rely on logical durations; instead, interval boundaries are defined by logical clocks (not necessarily harmonic). Figure 5 illustrates this.

LET-based languages generally assume strict periodic activation, but sLET relaxes this: the time intervals between logical timestamps may vary (Figure 6).

Communication obeys the *Visibility Principle* [28], but based on periodic updated of variables. Two means of communication are supported in PsyC: (1) Temporal Variables (TVs): shared variables periodically sampled on clocks. A TV sample is updated by the nearest producer EA whose termination precedes the sample timestamp; consumers read the nearest sample preceding its release date. Unlike HTL, sLET tasks are not defined by the samples they use.

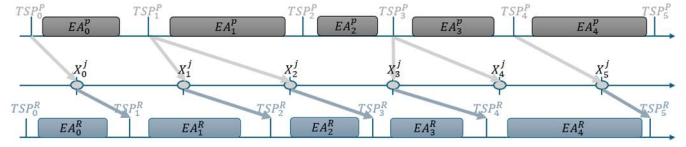


Figure 6. sLET communication through Temporal Variable

For example, EA_2^R consume data from EA_0^P , but EA_3^R cannot consume EA_1^P because the sampling occurs before EA_1^P termination. Under classical LET, the same configuration yields different dataflow (EA_3^R consume EA_1^P), highlighting a key difference between the two models.

(2) Streams: queues assigned to a consumer and updated by producers at predefined instants; producers push data, consumers pop data [27], [31].

3. METHODS AND PROCEDURE OF COMPARISON

The goal of this comparison is to guide developers in choosing the appropriate LET-based programming language. We evaluate the languages along three axes: (1) **Expressiveness**: the ability of each language to describe timing behavior and coordination mechanisms, assessed through a set of representative execution patterns. (2) **Industrial use cases**: typical application domains and their associated industrial requirements. (2) **Framework**: tool support and development environment, including debugging, scheduling, model-based visualization, tracing, and verification capabilities.

3.1. Comparison Based on Supported Execution Patterns

First, we examine the programming languages expressiveness through key execution patterns relevant for real-time systems.

1. Periodicity: (i) periodic: fixed-rate task execution. (ii) Sporadic: irregular events with a minimum inter-arrival time. (ii) Aperiodic: single, unpredictable events (not considered here).

All languages support periodic behavior. Because LET tasks are inherently periodic, Giotto, HTL, and TDL restrict themselves to periodic tasks. However, xGiotto supports sporadic patterns. In contrast, LF (SR-based) and PsyC (sLET-based) also support sporadic tasks, allowing more flexibility in handling asynchronous behaviors.

2. Modes: we distinguish four types of modes: (1) **Boot mode**: define which application starts when the system powers on. Switching between boot modes requires a full reboot of the hardware, so they are used only for large-scale operational changes (e.g., normal operation vs maintenance).
3. Hierarchy: refers to structuring programs into nested or layered components, enabling abstraction and modular verification.

HTL is fully hierarchical via *mode refinement*, where a mode may refine a full program. LF supports hierarchy through *composed reactors* but not nested modes. Giotto, TDL, PsyC do not offer hierarchical structuring of this form.

	Periodic	Sporadic	Hierarchy	Boot mode	Partition mode	Functional mode	System mode	Communication
Giotto	✓	✓	✗	✓	✓	✓	✓	Port
HTL	✓	✗	✓	✓	✓	✓	✓	Port; Communicator
TDL	✓	✗	✗	✓	✓	✓	✓	Port
LF	✓	✓	✓	✓	✓	✓	✓	Port
PsyC	✓	✓	✗	✓	✓	✓	✗	TV; Stream

Table 1. Comparison based on pattern support

4. **Communication Means:** languages rely on four mechanisms: (i) **Ports** (Giotto, HTL, TDL, LF): shared variables for inter-task communication. HTL restricts ports to intra-module use. Giotto and HTL use **drivers** to govern visibility, while LF and TDL automate this. (ii) **Communicators** (HTL): time-indexed ports used across modules. (iii) **Temporal Variables** (PsyC): LET-aligned shared variables analogous to communicators. (iv) **Streams** (PsyC): FIFO communication respecting sLET visibility rules.

These distinctions are organized in Table 1.

3.1.1. Flying model helicopter use case

This subsection introduces the helicopter controller used as a reference model. The original system [32], developed at ETH Zürich, is a centralized time-triggered autopilot running on a single StrongARM-based CPU [32], [33]. It is structured as a multi-mode, multi-rate controller: A fast task executes at 200 Hz (5 ms) for filtering and stabilization. A slow task executes at 40 Hz (25 ms) for navigation and control.

Modes determine how the slow component interprets the filtered data (e.g., initialization, idle, take-off, manual vs. autonomous flight). with deterministic, time-triggered communication between components (Figure 7).

For comparison purposes, we adopt a simplified version of this architecture. Instead of the full six-mode controller, we retain only what is needed to illustrate how each language expresses multi-rate, mode-dependent behavior: (i) Fast periodic task (5 ms): produces a filtered value. (ii) Slow periodic task (25 ms): computes control and data outputs. (iii) Two functional modes: ControlOff (manual-like) using NavPilot, ControlOn (autonomous) using NavControl. (iv) Deterministic mode switching, driven by a Boolean flag (flagControl). (v) Internal communication only (no sensors, actuators, or physical dynamics).

This part captures the key architectural elements—multi-rate execution, deterministic communication, and mode switching—while remaining small enough to re-implement consistently in Giotto, HTL, TDL, LF, and PsyC in the following sections.

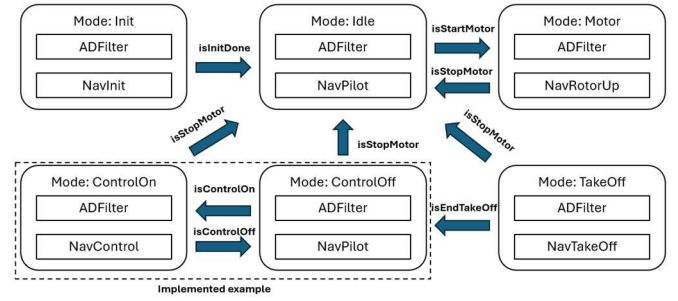


Figure 7. The operating modes of the controller [33].

3.1.2 Flying model helicopter design comparison

In this subsection, we model the helicopter controller using the languages discussed earlier. While implementations were developed for all of them (available at <https://github.com/asterios-technologies/ERTS26-paper16-examples.git>), we focus here on TDL, LF, and PsyC, as they currently benefit from industrial support. TDL closely mirrors the structure of Giotto and HTL, motivating our choice.

1. Lingua Franca

The helicopter controller is modeled in Lingua Franca as a single reactor with two timers and two modes. A 5 ms timer triggers a mode-independent reaction that executes the filtering stage by calling ADFilterImplementation and updating filterC. Because this reaction is defined outside any mode, it runs continuously regardless of the active control mode.

The control behavior is defined by two mutually exclusive modes, ControlOff and ControlOn, with the reactor starting in ControlOff. Each mode contains a reaction triggered by a 25 ms timer. The ControlOff mode executes NavPilotImplementation, while ControlOn executes NavControlImplementation. These reactions update the control and data states and may request a mode transition using `lf_set_mode`, which takes effect at the next logical time step. Transition conditions depend on the value of `flagControlC`, evaluated through `isControlOn` and `isControlOff`.

```

target C;
main reactor Helicopter {
  state filterC:int;
  state controlC:int;
  state dataC:int;
  state flagControlC:bool;
  timer fastTimer(0, 5 ms); // 200 Hz
  timer slowTimer(0, 25 ms); // 40 Hz
  reaction(startup) {
    filterC = InitFilter();
    controlC = InitControl();
    dataC = InitData();
    flagControlC = InitFlag(); // set initial mode command
  }
  reaction(fastTimer) {
    filterC = ADFilterImplementation(filterC);
  }
  initial mode ControlOff {
    // NavPilot runs every 25 ms in ControlOff.
    reaction(slowTimer) {
      NavPilotImplementation(filterC, &controlC, &dataC);
      // Switch request: effective at the next logical time step.
      if (isControlOn(flagControlC)) {
        If_set_mode(ControlOn);
      }
    }
  }
  mode ControlOn {
    reaction(slowTimer) {
      NavControlImplementation(filterC, &controlC, &dataC);
      if (isControlOff(flagControlC)) {
        If_set_mode(ControlOff);
      }
    }
  }
}

```

Code 1. LF program

2. TDL

The TDL model separates the controller into two modules that reflect the two execution rates of the system. The ADFilterModule, executed every 5 ms, contains the ADFilter task, which reads the current value of filterC, applies ADFilterImpl, and writes the updated value back. This module has a single mode (FastMode), ensuring that filtering runs periodically and independently of the controller mode.

The ControllerModule, executed every 25 ms, contains the variables filterC, controlC, dataC, and the boolean flagControl, which determines the active control mode. Two tasks define the control laws: NavPilot for the ControlOff mode and NavControl for the ControlOn mode. Each task reads filterC, computes new control outputs, and updates controlC and dataC.

The control logic is expressed through two periodic modes, ControlOff and ControlOn. In ControlOff, the module executes NavPilot and updates the control variables; at the end of the mode, the guard on flagControl selects whether to stay in ControlOff or switch to ControlOn. The ControlOn mode

mirrors this structure using NavControl. Only one mode is active at a time, and the guarded transitions ensure deterministic switching based on the system state.

```

/* ----- 5 ms Filtering Module (ADFilter) ----- */
module ADFilterModule {
  int filterC; // internal filtered value (5 ms update rate)
  /* Filtering task: reads and writes filterC every 5 ms */
  task ADFilter {
    input int f := filterC;
    output int newFilter := filterC;
    schedule ADFilterImpl(f, newFilter);
  }
  /* Mode */
  start mode FastMode [5ms] {
    task [1] ADFilter(filterC) [1]
    filterC := ADFilter.newFilter; // update filter value
    mode [1] FastMode(); // stay in same mode
  }
}
/* ----- 25 ms Controller Module ----- */
module ControllerModule {
  /* Internal communication variables (25 ms update rate) */
  int filterC, controlC, dataC;
  bool flagControl; // mode-switch flag
  // task invocation
  task NavPilot {
    input int f := filterC;
    output int newControl := controlC;
    output int newData := dataC;
    schedule NavPilotImpl(f, newControl, newData);
  }
  task NavControl {
    input int f := filterC;
    output int newControl := controlC;
    output int newData := dataC;
    schedule NavControlImpl(f, newControl, newData);
  }
  /* ----- Mode: ControlOff ----- */
  start mode ControlOff [25ms] {
    task [1] NavPilot(filterC) [1]
    controlC := NavPilot.newControl; // update outputs
    dataC := NavPilot.newData;
    // Switch to ControlOn
    mode [1] if flagControl then ControlOn() else ControlOff();
  }
  /* ----- Mode: ControlOn ----- */
  mode ControlOn [25ms] {
    task [1] NavControl(filterC) [1]
    controlC := NavControl.newControl;
    dataC := NavControl.newData;
    // Switch to ControlOff
    mode [1] if flagControl then ControlOff() else ControlOn();
  }
}

```

Code 2. TDL program

3. PsyC

The helicopter controller is modeled in PsyC using two agents, associated with two logical clocks, executed on parallel. The fast clock (`clk_fast`, 5 ms) drives the filtering stage, and the slow clock (`clk_slow`, 25 ms) drives the controller. Communication between agents is performed through temporal variables, which store values sampled according to the clocks that govern them. The `ADFilter` agent, triggered every 5 ms, implements the filtering logic. It reads the most recent value of `temporal_filter`, applies `ADFilterImplementation`, and writes the updated value back to the temporal variable. The controller agent is activated every 25 ms by the slow clock. At each activation, the agent reads the latest values of the temporal variables: the filtered sensor value (`temporal_filter`), the mode flag (`temporal_flagControl`), and the previous control outputs. A local Boolean variable `flag` represents the current control mode. The agent begins in the *start* body, where `temporal_flagControl` is initialized and the execution transitions immediately to the *ControlOff* body. The controller logic is defined through two bodies, *ControlOff* and *ControlOn*, only one of which is executed at each 25 ms activation, unlike agents that execute in parallel. In *ControlOff*, the agent reads the current filter, data, and control temporal variables, then calls `NavPilotImplementation` to update the temporal variables `temporal_control_update` and `temporal_data_update`. It also reads the mode flag, and if the flag indicates a transition, it switches to the *ControlOn* body using the next instruction. Similarly, in *ControlOn*, the agent reads the same temporal variables, executes `NavControlImplementation`, and updates the temporal variables accordingly. It then checks whether a return to *ControlOff* is required, and if so triggers the mode change. Each body ends with an advance statement, which completes the current logical execution and schedules the next activation at the following tick of the slow clock.

```
clock clk_fast = 5 * ast_realtime_ms; // 5 ms
clock clk_slow = 25 * ast_realtime_ms; // 25 ms
/* ----- Temporal Variables ----- */
temporal int temporal_filter with clk_fast;
temporal int temporal_control_update with clk_slow;
temporal int temporal_data_update with clk_slow;
temporal bool temporal_flagControl with clk_slow;
/* ----- FAST AGENT (5 ms) ----- */
agent ADFilter(uses realtime, starttime 1 with clk_fast) {
  display temporal_filter:all;
  consult 1 $ temporal_filter;
  body start {
    int prev = $[0]temporal_filter;
    temporal_filter = ADFilterImplementation(prev);
    advance 1 with clk_fast;
  }
}
```

Code 3. PsyC program — Part 1: Agent ADFilter

```
/* ----- CONTROLLER AGENT (25 ms) ----- */
/* Local variables initialization */
int control_update = 0, data_update = 0, filterVal = 0;
agent Controller(uses realtime, starttime 1 with
clk_slow) {
  /* Display TVs to all agents */
  display {
    temporal_control_update:all;
    temporal_data_update:all;
    temporal_flagControl:all;
  }
  /* Controller can consult these TVs */
  consult {
    1 $ temporal_filter;
    1 $ temporal_flagControl;
    1 $ temporal_data_update;
    1 $ temporal_control_update;
  }
  /* Internal state used for mode switching */
  global { bool flag = false; }
  /* ----- Start Body ----- */
  body start {
    temporal_flagControl = true; // initialize mode flag
    next ControlOff; // enter ControlOff mode
  }
  /* ----- ControlOff Mode ----- */
  body ControlOff {
    /* Read latest sampled values from TVs */
    filterVal = $[0]temporal_filter;
    flag = $[0]temporal_flagControl;
    // Mode-switch condition:
    if (flag) {
      next ControlOn;
    }
    // Execute pilot controller
    NavPilotImplementation(filterVal, &control_update,
&data_update);
    // Update control and data values
    temporal_control_update = control_update;
    temporal_data_update = data_update;
    // Proceed to next 25 ms activation
    advance 1 with clk_slow;
  } /* ----- ControlOn Mode ----- */
  body ControlOn {
    filterVal = $[0]temporal_filter;
    flag = $[0]temporal_flagControl;
    if (!flag) {
      next ControlOff;
    }
    NavControlImplementation(filterVal,
&control_update, &data_update);
    temporal_control_update = control_update;
    temporal_data_update = data_update;
    advance 1 with clk_slow;
  }
}
```

Code 4. PsyC program — Part2: Agent Controller

	Year of introduction	Supported paradigm	Real-time constraints	Use case/ case study	Industrial support
Giotto [7], [11], [33]	2001	LET	Hard	Flying model helicopter [32]	?
HTL [13]—[15]	2002	LET	Hard	flying model helicopter [32]	?
TDL [8], [16]—[19]	2003	LET	Hard	Active Rear Steering system [17]	CHRONA
LF [22]—[27]	2020	SR	Soft	distributed control robotic arm [37]	XRONOS
PsyC [28]—[32]	1998	sLET	Hard	Certified avionics systems [38]	Asterios Technologies

Table 2. Comparison based on LET integration and implementation context

3.1. Comparison based on industrial use cases

These languages also target different domains. Giotto, HTL, TDL, and PsyC focus on hard real-time, safety-critical systems, while LF is oriented toward soft real-time, distributed, or cyber-physical systems. Applications include the ETH Zürich autonomous helicopter [32] (Giotto [33] HTL [5]). TDL has been applied in the automotive domain, such as Active Rear

Steering (ARS [17]) systems, supported today by the CHRONA toolchain. PsyC tool chain is DO-330 qualified for DO-178C DAL A aerospace and defense applications (engine regulation, navigation, landing systems, and flight-control systems) and is provided by Asterios Technologies.

LF, developed more recently, has been adopted in distributed robotics [37], cloud and edge computing [21], [35] and cyber-physical coordination scenarios, supported by the XRONOS toolchain.

Although Giotto, HTL, TDL, LF, and PsyC all rely on LET concepts, their implementations differ. HTL and TDL remain close to Giotto’s original definition. LF integrates LET semantics into a SR framework, and PsyC adopts the sLET model [30].

Table 2 summarizes the key points discussed in subsection 3.2.

3.2. Comparison based on framework

While Sections 3.1 and 3.2 examined the languages in terms of expressiveness, application types, and industrial use cases, tool support remains a decisive factor for selecting a suitable language. In practice, the availability of development tools strongly influences how easily developers can model, debug, and validate real-time systems. This section therefore compares toolchains of languages considered along several aspects relevant to industrial practice.

1. Framework: LF and TDL are integrated into Eclipse-based IDE, PsyC is embedded into a web-based IDE, and Giotto/HTL debugging is supported through academic tools.

2. Scheduling: Giotto, HTL, TDL, and LF rely on dynamic scheduling (EDF-like). PsyC uses static scheduling computed offline, thanks to a qualified toolchain.
3. Model-based design: Giotto, HTL, TDL, and LF support model-based design through environments such as Simulink, Eclipse, or VS Code plugin. PsyC is not yet model-based supported, but it is currently in the next release.
4. Execution profiling: TDL, LF, and PsyC offer built-in execution profiling to visualize timing behavior.
5. Licensing: TDL and PsyC are distributed under end-user licenses, whereas Giotto, HTL, and LF are distributed under open licenses.

4. CONCLUSION

This paper presented a comparative analysis of several programming languages based on the logical time principles. Although these languages share the common goal of ensuring deterministic behavior in real-time systems, our study shows that they differ significantly in expressiveness, tooling, execution models, and intended domains of application. Consequently, there is no universally “best” LET-based language. Instead, each one is tailored to a specific ecosystem and set of constraints.

Giotto and HTL offer clean theoretical foundations and strong time-deterministic guarantees, making them valuable for academic studies, and studies on LET semantics or timing refinement.

TDL, with its stricter semantics and automated toolchain, is better aligned with *automotive and industrial control* use cases, where predictable periodic behavior and simplified developer tooling are essential. Lingua Franca targets *soft real-time and distributed applications*, providing modern abstractions, polyglot integration, and a flexible synchronous-reactive model suitable for cyber-physical systems, edge computing, and research prototypes.

Finally, PsyC, built on the sLET paradigm, is designed for *certifiable hard real-time systems*, with a fully deterministic execution model, static scheduling, and a toolchain compliant

with DO-178C DAL-A constraints. It is therefore suited for avionics, space, and other highly safety-critical domains.

Overall, the diversity of these languages illustrates that LET is a common answer for a better quality and robustness of embedded systems. So, it is not tied to a single programming style but can be implemented through multiple abstractions—modes, modules, reactors, agents, or pure logical clocks. The appropriate choice depends not on theoretical elegance but on the system’s requirements, its certification context, and the surrounding development toolchain.

REFERENCES

- [1] H. Kopetz and W. Steiner, *Real-Time Systems: Design Principles for Distributed Embedded Applications*. Springer Nature, 2022.
- [2] J.-L. Bergerand, ‘LUSTRE: un langage déclaratif pour le temps réel’, Institut National Polytechnique de Grenoble-INPG, 1986.
- [3] N. Halbwachs, ‘Synchronous Programming of Reactive Systems’.
- [4] K.-B. Gemlau, L. Köhler, R. Ernst, and S. Quinton, ‘System-level Logical Execution Time: Augmenting the Logical Execution Time Paradigm for Distributed Real-time Automotive Software’, *ACM Trans. Cyber-Phys. Syst.*, vol. 5, no. 2, pp. 1–27, Apr. 2021, doi: 10.1145/3381847.
- [5] C. M. Kirsch and A. Sokolova, ‘The Logical Execution Time Paradigm’, in *Advances in Real-Time Systems*, S. Chakraborty and J. Eberspächer, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 103–120. doi: 10.1007/978-3-642-24349-3_5.
- [6] V. David, J. Delcoigne, E. Leret, A. Ourghanlian, P. Hilsenkopf, and P. Paris, ‘Safety Properties Ensured by the OASIS Model for Safety Critical Real-Time Systems’, in *Computer Safety, Reliability and Security*, vol. 1516, W. Ehrenberger, Ed., in Lecture Notes in Computer Science, vol. 1516, Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 45–59. doi: 10.1007/3-540-49646-7_4.
- [7] T. A. Henzinger, B. Horowitz, and C. M. Kirsch, ‘Giotto: a time-triggered language for embedded programming’, *Proc. IEEE*, vol. 91, no. 1, pp. 84–99, Jan. 2003, doi: 10.1109/JPROC.2002.805825.
- [8] E. Farcas, C. Farcas, W. Pree, and J. Templ, ‘Transparent Distribution of Real-Time Components Based on Logical Execution Time’, *Proceedings of the 2005 ACM SIGPLAN/SIGBED conference on Languages, Compilers, and Tools for Embedded Systems*, pp. 31–39, 2005.
- [9] S. Wang *et al.*, ‘Optimizing Logical Execution Time Model for Both Determinism and Low Latency’, Mar. 07, 2024, *arXiv*: arXiv:2310.19699. doi: 10.48550/arXiv.2310.19699.
- [10] A. Biondi, P. Pazzaglia, A. Balsini, and M. Di Natale, ‘Logical execution time implementation and memory optimization issues in autosar applications for multicores’, in *International Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems (WATERS)*, 2017.
- [11] T. A. Henzinger, B. Horowitz, and C. M. Kirsch, ‘Embedded Control Systems Development with Giotto*’, *Proceedings of the ACM SIGPLAN workshop on Languages, compilers and tools for embedded systems*, pp. 64–72, 2001.
- [12] M. A. Sanvido, A. Ghosal, and T. A. Henzinger, *XGIOTTO language report*. Computer Science Division, University of California, 2003.
- [13] I. D. Iercan, ‘CONTRIBUTIONS TO THE DEVELOPMENT OF REAL-TIME PROGRAMMING TECHNIQUES AND TECHNOLOGIES’, *Timișoara: Editura Politehnica*, 2008.
- [14] A. Ghosal, A. Sangiovanni-Vincentelli, C. M. Kirsch, T. A. Henzinger, and D. Iercan, ‘A hierarchical coordination language for interacting real-time tasks’, in *Proceedings of the 6th ACM & IEEE International conference on Embedded software - EMSOFT ’06*, Seoul, Korea: ACM Press, 2006, p. 132. doi: 10.1145/1176887.1176907.
- [15] A. Ghosal, D. Iercan, C. M. Kirsch, T. A. Henzinger, and A. Sangiovanni-Vincentelli, ‘Separate compilation of hierarchical real-time programs into linear-bounded Embedded Machine code’, *Science of Computer Programming*, vol. 77, no. 2, pp. 96–112, Feb. 2012, doi: 10.1016/j.scico.2010.06.004.
- [16] J. Templ, ‘TDL Specification and Report’.
- [17] W. Pree and J. Templ, ‘Modeling with the Timing Definition Language (TDL)’, in *Model-Driven Development of Reliable Automotive Services*, vol. 4922, M. Broy, I. H. Krüger, and M. Meisinger, Eds., in Lecture Notes in Computer Science, vol. 4922, Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 133–144. doi: 10.1007/978-3-540-70930-5_9.
- [18] S. Resmerita, P. Derler, and W. Pree, ‘Timing Definition Language (TDL) Modeling in Ptolemy II’, *Department of Computer Science, University of Salzburg, Tech. Rep.*, 2008.
- [19] T. Kloda, ‘Towards a More Flexible Timing Definition Language’, *12th International Workshop Quantitative Aspects of Programming Languages and Systems-at ETAPS 2014*, 2014.
- [20] T. Kloda, B. d’Ausbourg, and L. Santinelli, ‘EDF schedulability analysis for an extended Timing Definition Language’, in *Proceedings of the 9th IEEE International Symposium on Industrial Embedded Systems (SIES 2014)*, Pisa: IEEE, Jun. 2014, pp. 30–40. doi: 10.1109/SIES.2014.6871183.
- [21] M. Lohstroh, C. Menard, S. Bateni, and E. A. Lee, ‘Toward a Lingua Franca for Deterministic Concurrent Systems’, *ACM Trans. Embed. Comput. Syst.*, vol. 20, no. 4, pp. 1–27, Jul. 2021, doi: 10.1145/3448128.
- [22] H. M. F. Lohstroh, ‘Reactors: A Deterministic Model of Concurrent Computation for Reactive Systems’, *University of California, Berkeley*, 2020.
- [23] A. Schulz-Rosengarten, R. Von Hanxleden, M. Lohstroh, E. A. Lee, and S. Bateni, ‘Polyglot Modal Models through

- Lingua Franca’, in *Proceedings of Cyber-Physical Systems and Internet of Things Week 2023*, San Antonio TX USA: ACM, May 2023, pp. 337–342. doi: 10.1145/3576914.3587498.
- [24] E. A. Lee and M. Lohstroh, ‘Generalizing Logical Execution Time’, in *Principles of Systems Design*, vol. 13660, J.-F. Raskin, K. Chatterjee, L. Doyen, and R. Majumdar, Eds., in Lecture Notes in Computer Science, vol. 13660, Cham: Springer Nature Switzerland, 2022, pp. 160–181. doi: 10.1007/978-3-031-22337-2_8.
- [25] ‘Introduction | Lingua Franca’. Accessed: Nov. 28, 2025. [Online]. Available: <https://www.lf-lang.org/docs/>
- [26] A. Benveniste and G. Berry, ‘The synchronous approach to reactive and real-time systems’, *Proc. IEEE*, vol. 79, no. 9, pp. 1270–1282, Sep. 1991, doi: 10.1109/5.97297.
- [27] F. Siron and D. Potop-Butucaru, ‘Formal Semantics of the PsyC language’, *Inria-Sophia Antipolis*, 2023.
- [28] D. Chabrol *et al.*, ‘Separation of functional and time interference concerns for efficient AMC 20-193 compliance’, *ERTS2024.*, 2024.
- [29] D. Chabrol, ‘Etude, conception et mise en oeuvre d’un protocole de communication synchrone tolérant aux fautes et prédictible sur des composants réseaux standards’, Paris 11, 2006.
- [30] F. Siron and D. Potop-Butucaru, ‘The synchronous Logical Execution Time paradigm’.
- [31] F. Siron, ‘Méthodologie de vérification formelle de propriétés temporelles pour les applications temps-réel critiques basées sur le concept de temps logique’.
- [32] J. Chapuis, C. Eck, M. Kottmann, M. Sanvido, and O. Tanner, ‘Control of Helicopters’, in *Control of Complex Systems*, K. Åström, P. Albertos, M. Blanke, A. Isidori, W. Schaufelberger, and R. Sanz, Eds., London: Springer London, 2001, pp. 359–392. doi: 10.1007/978-1-4471-0349-3_16.
- [33] C. M. Kirsch, M. A. A. Sanvido, T. A. Henzinger, and W. Pree, ‘A Giotto-Based Helicopter Control System’, in *Embedded Software*, vol. 2491, A. Sangiovanni-Vincentelli and J. Sifakis, Eds., in Lecture Notes in Computer Science, vol. 2491, Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 46–60. doi: 10.1007/3-540-45828-X_5.
- [34] T. A. Henzinger, C. M. Kirsch, E. R. B. Marques, and A. Sokolova, ‘Distributed, Modular HTL’, in *2009 30th IEEE Real-Time Systems Symposium*, Washington DC, USA: IEEE, Dec. 2009, pp. 171–180. doi: 10.1109/RTSS.2009.9.
- [35] R. Von Hanxleden *et al.*, ‘Pragmatics Twelve Years Later: A Report on Lingua Franca’, in *Leveraging Applications of Formal Methods, Verification and Validation. Software Engineering*, vol. 13702, T. Margaria and B. Steffen, Eds., in Lecture Notes in Computer Science, vol. 13702, Cham: Springer Nature Switzerland, 2022, pp. 60–89. doi: 10.1007/978-3-031-19756-7_5.
- [36] C. Menard *et al.*, ‘High-performance Deterministic Concurrency Using LINGUA FRANCA’, *ACM Trans. Archit. Code Optim.*, vol. 20, no. 4, pp. 1–29, Dec. 2023, doi: 10.1145/3617687.
- [37] H. Austad and G. Mathisen, ‘Bounding the End-to-End Execution Time in Distributed Real-Time Systems: Arguing the case for Deterministic Networks in Lingua Franca’, in *Proceedings of Cyber-Physical Systems and Internet of Things Week 2023*, San Antonio TX USA: ACM, May 2023, pp. 343–348. doi: 10.1145/3576914.3587499.
- [38] A. Methni, E. Ohayon, and F. Thureau, ‘ASTERIOS Checker: A Verification Tool for Certifying Airborne Software’.